CSED233 · 데이터구조 · 포스텍

정리본 1. Introduction

자료구조가 무엇인지, 왜 선택이 성능을 좌우하는지, 이 강의에서 무엇을 배우는지.

1. 자료구조(Data Structure)란?

자료구조(Data Structure): 데이터를 조직하고 효율적으로 관리하는 방법.

Program = Data Structure + Algorithm

프로그램의 성능은 알고리즘의 효율, 자료구조의 연산 특성, 입력 규모, 메모리 접근 패턴, I/O 및 실행 환경 등 여러 요인의 영향을 받는다. 특히 자료구조 선택이 부적절하면 검색·삽입·삭제 등의 연산 비용이 크게 증가할 수 있다. 따라서 문제의 주요 연산과 요구사항을 분석하여 적절한 자료구조와 알고리즘을 함께 선택해야 한다. 실제 성능 병목은 측정과 분석을 통해 확인한다.

- **Data Structure(자료구조)**는 "데이터를 어떻게 저장할 것인가"를 결정한다 → 메모리 구조, 연산 방식, 성능 특성.
- **Algorithm(알고리즘)**은 "데이터를 어떻게 처리할 것인가"를 결정한다 → 단계별 절차, 연산 조합, 최적화 방식.

둘은 독립적이지 않다. 자료구조는 알고리즘의 구현 방식과 연산 효율에 영향을 주지만, 사용할 수 있는 알고리즘을 하나로 결정하지는 않는다.

예제 · 자료구조가 알고리즘을 제한한다

같은 "정렬" 작업도 자료구조에 따라 선택지가 달라진다.

- **배열(Array)**: Bubble, Selection, Insertion, Merge, Quick, Heap — 6가지 이상 선택 가능.
- **연결 리스트(Linked List)**: 임의 위치의 노드를 찾으려면 O(n) 순회가 필요해, 배열의 인덱스 기반 Quick Sort 구현을 그대로 적용하면 비효율적이다. Merge Sort가 대표적으로 적합하며 최악 O(n log n)을 보장한다. Insertion Sort도 구현할 수 있지만 일반적으로 평균·최악 $O(n^2)$이다.

- **Binary Search Tree(이진 탐색 트리)**: In-order Traversal(중위 순회)은 키를 오름차순으로 방문하는 것이지 트리를 정렬하는 연산은 아니다 (O(n)).

→ 자료구조 선택은 알고리즘의 구현 방식과 효율에 영향을 주지만, 사용할 수 있는 알고리즘을 하나로 결정하지는 않는다.

2. 자료구조의 종류



이 강의에서 다루는 자료구조의 분류

2.1 선형 자료구조

배열(Array)



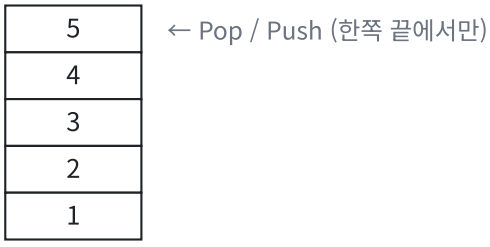
메모리 연속 할당 · Random Access O(1) ✓ · 크기 고정 ✗

연결 리스트(Linked List)

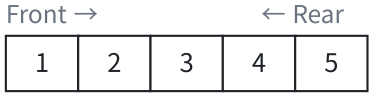


메모리 분산 할당 · 크기 유동적 ✓ · Random Access 불가 ✗

스택(Stack) — LIFO



큐(Queue) — FIFO

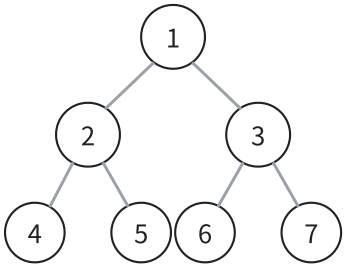


삽입은 Rear(뒤) · 삭제·처리는 Front(앞)에서

선형 자료구조 4종: 같은 1차원이지만 접근 방식이 다르다

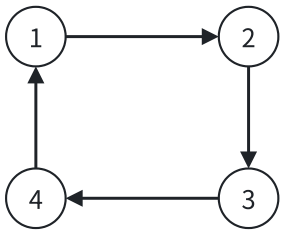
2.2 비선형 자료구조

트리(Tree) — 계층적



부모-자식 관계 · 순회 방식 다양 · BST는 inorder로 정렬된 순서 획득 가능

그래프(Graph) — 복잡한 관계



정점(노드)과 간선 · 방향/무방향 · 최단경로, 연결성 분석

해시 테이블(Hash Table)



Hash Function으로 Key → 인덱스 변환
평균 O(1) 검색 · 키의 정렬 순서는 보장

비선형 자료구조 3종

3. 왜 중요한가: 100만 개 검색 실험

100만 개 데이터에서 특정 값 하나를 찾는다고 하자. 자료구조만 바꿨을 뿐인데 결과는 이렇게 갈린다 (연산당 1ns 가정).

자료구조	시간복잡도	실제 비교 횟수	소요 시간
배열 + 선형 탐색	O(n)	평균 500,000회	0.5ms
이진 탐색 트리(BST, 균형)	O(log n)	평균 20회	0.00002ms
해시 테이블	O(1)	평균 1회	0.000001ms

- 배열 vs BST → **25,000배** 차이, 배열 vs Hash → **500,000배** 차이.
- 작은 규모에서는 절대적인 경계가 없고 상수·구현 비용이 더 중요할 수 있지만, n이 커지면 O(n)과 O(log n) 검색 횟수 차이의 비율이 n/log n 형태로 커진다. 비율은 약 $n/(2\log_2 n)$으로, n = 100K에서 약 3,010배, n = 1M에서 약 25,086배이다.

같은 연산도 구조마다 비용이 다르다

"중간에 삽입" 한 번을 예로 들면:

- **배열**: 인덱스가 주어지면 접근 O(1) + 오른쪽 원소 Shift O(n) = O(n).

- **연결 리스트**: 위치까지 순회하면 O(n)이 들고, 직전 노드 포인터가 주어지면 삽입 자체는 O(1)이다. 연결 위치를 이미 알 때 배열보다 유리하다.
- **트리**에서 찾기: 미정렬 배열의 선형 탐색은 평균 O(n), 정렬 배열의 이진 탐색은 O(log n), 균형 BST는 O(log n), 일반 BST는 최악 O(n)이다. n = 1M 기준 미정렬 배열 평균 약 500,000회 vs 균형 BST 약 20회로 약 25,000배이다.

문제 해결 방식 자체가 자료구조에 의존한다.

4. 예제로 보는 선택의 차이: 성적 조회 시스템

요구사항: 학생 1000명, 학번을 입력하면 성적 반환. 같은 요구사항을 세 가지 자료구조로 설계해 보자.

방안 비교

방안	구조	검색 방식	평균 비교
A. 배열	[성적1][성적2]…[성적1000]	처음부터 끝까지 순서대로 비교	O(n) ≈ 500회
B. BST	중앙값 학번 500이 루트, 좌우로 분기 (균형 트리 가정)	크기 비교로 절반씩 제외	O(log n) ≈ 10회
C. 해시 테이블	Hash Function으로 학번 → 인덱스	계산 후 바로 접근	O(1) ≈ 1회

코드 예시

```
// A. 배열
for (i = 0; i < 1000; i++) if (id[i] == target) return score[i];

// B. BST
if (node == nullptr) return NOT_FOUND;
if (target == node->key) return node->score;
if (target < node->key) return search(node->left);
else return search(node->right);

// C. 해시 테이블
return hashTable[hash(target)];
```

해시 테이블의 약 1회 접근은 충돌이 없는 이상적 모델 기준이다. 실제로는 충돌 처리와 해시 계산 비용이 있을 수 있지만, 적절한 가정 아래 평균 O(1)이다.

같은 문제인데 **500회 vs 10회 vs 1회** — 자료구조만으로 50배 이상 차이가 난다.

5. 어떤 자료구조를 고를까

정답이 하나로 정해진 것이 아니라, 문제의 조건을 보고 트레이드오프를 따지는 과정이다. 세 가지 질문을 순서대로 던지면 된다.

선택 기준 3단계

1. **가장 빈번한 연산이 무엇인가?**
   - 검색 위주 → Hash Table / BST (예: 전화번호부, 성적 조회)
   - 최대/최소가 자주 필요 → Heap (예: 작업 우선순위, CPU 스케줄링)
   - 삽입/삭제 + 순서 유지 → AVL Tree (예: 순위 변동, 실시간 업데이트). AVL이 유지하는 것은 키의 정렬 순서이다.
2. **메모리 제약이 있는가?**
   - 제한 있음 → 메모리 사용량을 따져 선택한다. 연결 리스트도 노드마다 포인터 오버헤드가 있어 메모리 제한만으로 연결 리스트를 고르지는 않는다 (hash table처럼 여유 bucket이 불필요하다는 점은 비교 요소. 예: 임베디드)
   - 충분함 → Hash Table (예: 클라우드, 고성능 서버)
3. **최악의 경우 보장이 중요한가?**
   - 중요(실시간) → Merge Sort의 최악 O(n log n) 보장 — 보통 O(n) 추가 공간이 필요하고, 실시간에서는 실제 시간 상한도 확인해야 한다 (예: 의료, 항공, 금융)
   - 평균만 중요 → Quick Sort 평균 O(n log n), 최악 $O(n^2)$ - 피벗·입력에 따라 달라짐 (예: 일반 데이터 처리)

5.1 연산별 성능 요약

자료구조	검색	삽입	삭제	순서 유지	사용 시기

배열	O(1) (인덱스 접근), 정렬 시 값 검색 O(log n)	O(n), 끝은 분할상환 O(1) (재할당 시 최악 O(n))	O(n), 끝은 O(1)	○ (인덱스)	정렬된 고정 데이터
연결 리스트	O(n)	O(1) (위치 알 때)	O(1) (필요한 포인터를 알고 있을 때)	○ (순서)	크기가 자주 변할 때
BST	O(log n) 평균	O(log n) 평균	O(log n) 평균	○ (In-order)	검색 + 순서 유지
Hash Table	O(1) 평균	O(1) 평균	O(1) 평균	×	검색만 중요할 때
Heap	O(n)	O(log n)	O(log n) (루트 삭제 기준)	△ (부모-자식 부분 순서만)	우선순위 큐, Sort

BST의 '평균'은 랜덤 삽입 순서의 기대값; 편향되면 최악 O(n). AVL Tree 등 균형 BST는 검색·삽입·삭제 O(log n) 보장.

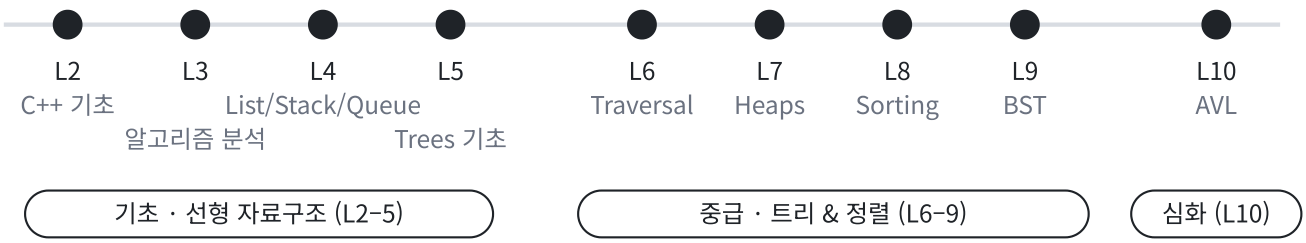
배열 모델 구분: 정적 배열은 크기가 고정되어 남는 용량이 있을 때만 끝 삽입이 O(1)이고 가득 차면 확장할 수 없다. 동적 배열은 용량이 부족하면 재할당·복사로 확장하며, 끝 삽입은 분할상환 O(1)이다 (재할당 회차에는 최악 O(n)).

5.2 n = 100만일 때 체감

자료구조	검색 1회	삽입 1회	체감
배열	평균 약 500,000회	중간 삽입 시 이동 평균 약 500,000개 (끝이면 1회)	매우 느림
연결 리스트	평균 약 500,000개 노드 방문	1회 (위치 알 때)	느림
BST (균형)	20회	20회	매우 빠름
해시 테이블	1회	1회	극도로 빠름

기준은 성공 검색 평균이다.

6. 강의 로드맵과 학습 전략



L2(C++)와 L3(알고리즘 분석)이 모든 것의 토대. L3, L8, L10에서 개념적 도약이 온다.

- **L2 C++ 기초**: 포인터, 동적 할당, 메모리 관리 — 자료구조 구현의 기초.
- **L3 알고리즘 분석**: Big-O 표기법, 시간/공간 복잡도 — 성능 평가의 기준.
- **L4–5**: 선형 자료구조 3종, 트리 기본 성질.
- **L6–9**: 순회와 트리 재구성, 우선순위 큐/힙, 정렬 5종, BST.
- **L10**: AVL Tree — Self-Balancing(자가 균형)과 Rotation(회전).

6.1 학습 목표 4단계

1. **개념 이해** — List, Stack, Queue, Tree가 무엇인지 정의를 안다.
2. **기본 연산 구현** — Insert, Delete, Search를 코드로 쓴다.
3. **성능 분석** — 각 연산의 시간/공간 복잡도를 Big-O로 계산한다.
4. **선택과 적용** — 주어진 문제에 어떤 자료구조를 쓸지, 왜 그것인지 설명한다. 이 강의의 최종 목표.

6.2 AI 활용 기준

- **허용**: 개념 질문("Stack이 뭐야?"), 구현 예시 요청, 강의 복습, 디버깅.
- **금지**: 시험 중 사용, AI가 쓴 코드 그대로 제출, 이해 없이 복사.
- "어떤 자료구조를 써야 하는가"를 판단하는 능력이 진짜 경쟁력이다.

7. Quiz

Q1. 같은 데이터로 배열과 BST를 구성했을 때, 검색 성능이 다른 이유는?

힌트

핵심은 "다음에 어디를 볼지 아는가"다. 배열은 원소의 위치 정보가 없어 처음부터 훑어야 하고, BST는 크기 비교 한 번으로 탐색 범위를 절반으로 줄인다.

정답

개념은 힌트 참고.

① 배열: 정렬되어 있지 않고 인덱스도 모르는 배열에서는 target이 어디 있는지 알 수 없어 평균 n/2회 비교 → O(n). 정렬된 배열이면 이진 탐색 O(log n), 인덱스를 알면 O(1)이다.

② BST: 균형 BST에서는 루트와 비교해 좌/우 하나를 버리는 것을 반복 → O(log n). 일반 BST는 최악 O(n)이다.

③ 같은 데이터인데 구조가 다르면 쓸 수 있는 탐색 절차가 달라진다.

정답: 자료구조가 연산 방식을 결정하기 때문 — 정렬·인덱싱되지 않은 배열은 선형 탐색 O(n), 균형 BST는 이진 탐색 O(log n)이 가능하다.

Q2. "자료구조 선택이 알고리즘을 결정한다"는 무슨 의미인가?

힌트

섹션 1의 정렬 예시를 떠올려 보자. 배열에서는 6가지 정렬이 가능하지만 연결 리스트에서는 QuickSort가 Random Access 부재로 비효율적이고, BST에서는 inorder 순회로 키를 오름차순으로 읽을 수 있다.

정답

개념은 힌트 참고.

① 배열 → Random Access 가능 → Bubble/Quick/Merge 등 대부분의 정렬 사용 가능.

② 연결 리스트 → 임의 위치 접근에 O(n) 순회가 필요해 배열식 Quick Sort는 비효율적이고, Merge Sort가 대표적으로 적합하다 (Insertion Sort는 평균·최악 $O(n^2)$).

③ BST → 키 순서 조건이 유지된 상태라면 In-order Traversal로 키를 오름차순 출력할 수 있고 순회 자체는 O(n)이지만, 이는 정렬 연산이 아니라 이미 유지된 순서를 읽는 것이다. BST를 만드는 과정은

별도다 (임의 입력 삽입부터 세면 균형 BST $O(n \log n)$, 일반 BST 최악 $O(n^2)$).

정답: 같은 문제라도 자료구조에 따라 효율적인 방식이 달라질 뿐, 하나의 알고리즘만 쓸 수 있는 것은 아니다. 자료구조를 고르면 효율적인 알고리즘 후보가 좁혀진다는 뜻이다.

Q3. 왜 배열과 연결 리스트가 둘 다 필요한가? 연결 리스트가 더 나은가?

> 힌트

각각의 장점이 상충(trade-off)한다: Random Access $O(1)$ ↔ 동적 크기.

> 정답

개념은 힌트 참고.

① Random Access가 필요하면 배열이 유리하다 — 인덱스 접근 $O(1)$이고, 동적 배열도 확장이 가능하다.

② 중간 삽입·삭제가 잦으면 연결 리스트가 유리할 수 있지만, 위치를 찾는 비용과 노드마다 붙는 포인터 오버헤드를 함께 따져야 한다.

③ 어느 한쪽이 모든 상황에서 우월하지 않다.

정답: 아니다. 둘은 트레이드오프 관계라 문제의 특성에 따라 골라야 한다.

Q4. 100만 개 데이터 검색에서 배열 vs BST 차이가 수만 배가 되는 이유는?

> 힌트

$O(n)$과 $O(\log n)$에 $n = 1{,}000{,}000$을 대입해 보자. $\log_2(10^6) \approx 20$.

> 정답

개념은 힌트 참고.

① 배열 선형 탐색: 평균 $n/2 = 500{,}000$회 비교.

② BST 이진 탐색: $\log_2$(100만) ≈ 20회 비교.

③ 500,000 / 20 = 25,000 → 약 2만 5천 배 차이.

정답: O(n) = 50만 회 vs O(log n) = 20회. 데이터가 클수록 차이는 더 벌어지므로 빅데이터에서는 자료구조가 성능을 사실상 결정한다.

Q5. Hash Table이 검색 O(1)인데도 모든 곳에 쓰이지 않는 이유는?

> 힌트

O(1)의 대가를 생각해 보자: 해시 충돌(Collision), 순서 정보 소실, 메모리 오버헤드.

> 정답

개념은 힌트 참고.

① Collision 처리 로직이 복잡하다.

② 데이터의 순서를 유지하지 못한다 → 순서가 필요하면 BST.

③ 메모리 오버헤드가 있다 → 메모리가 빠듯하면 배열.

정답: Collision 처리 비용, 순서 없음, 메모리 오버헤드 때문에 요구사항에 따라 BST나 배열이 더 나은 선택이 된다.